

D5.5.2.4 Integrated Text to Speech Conversion

Due date: **21/03/2025**
Submission Date: **21/03/2025**
Revision Date: **08/05/2026**

Start date of project: **01/07/2023**

Duration: **36 months**

Lead organisation for this deliverable:

Responsible Person: **Richard Muhirwa**

Revision: **1.4**

Project funded by the African Engineering and Technology Network (Afretec) Inclusive Digital Transformation Research Grant Programme		
Dissemination Level		
PU	Public	PU
PP	Restricted to other programme participants (including Afretec Administration)	
RE	Restricted to a group specified by the consortium (including Afretec Administration)	
CO	Confidential, only for members of the consortium (including Afretec Administration)	

Executive Summary

Deliverable D5.5.2.4 presents the outcomes of Task 5.5.2.4, which integrates the outputs of Tasks 5.5.2.1 (English Text-to-Speech) and 5.5.2.3 (Kinyarwanda Text-to-Speech) into a unified system. This document outlines the results of each stage of the software development process, covering requirements definition, module specification, interface design, module design, testing, and implementation. The Integrated Text-to-Speech system enables the Pepper robot to transform written text in either English or Kinyarwanda into spoken words. This capability is fundamental to the robot's ability to verbally communicate with users in multiple languages, supporting a wide range of interactions from basic greetings to complex information delivery. Both languages are synthesised by neural TTS models running on the host machine: a fine-tuned YourTTS model for Kinyarwanda and an XTTS v2 model for English. The generated audio can be routed to the robot for playback through the NAOqi ALAudioPlayer (`playback_mode = naoqi`) or played directly on the host's audio device for testing without a robot (`playback_mode = local`). The system exposes two ROS interfaces on the same name `/textToSpeech/say_text`: a traditional ROS *service* interface for short, blocking text-to-speech requests, and a ROS *action server* interface that provides asynchronous goals, real-time progress feedback, and the ability to preempt long utterances. The active interface is selected through the `interface` parameter in the configuration file (`service`, `action`, or `both`). This report details the functional requirements, interface design specifications, module architecture, testing approach, and implementation instructions for the integrated Text-to-Speech conversion system. Testing results confirm that the Pepper robot's multilingual TTS system functions correctly, producing clear speech in both English and Kinyarwanda for a wide range of inputs, across both the service and action server interfaces, and maintaining stability even under stress conditions.

Contents

1	Introduction	5
2	Requirements Definition	6
2.1	Overview	6
2.2	Functional Specification	6
2.3	Text Processing and Speech Synthesis	6
2.4	Configuration Management	8
2.5	Inputs and Outputs	8
2.5.1	Service Provided	8
2.5.2	Action Server Provided	8
2.6	External Dependencies and Integration	9
2.7	Error Handling and Logging	9
3	Function specification	11
3.1	Functional Characteristics	11
3.1.1	Text-to-Speech Conversion	11
3.1.2	Operation Modes	11
3.1.3	Audio File Generation and Playback	11
3.1.4	ROS Integration	11
3.2	Inputs and Outputs	11
3.2.1	Inputs	11
3.2.2	Outputs	12
3.3	Dependencies	12
3.3.1	Common Dependencies	12
3.3.2	English TTS Dependencies	12
3.3.3	Kinyarwanda TTS Dependencies	13
3.4	Execution Workflow	14
3.5	System Requirements	14
3.6	Limitations and Assumptions	15
4	Interface Design	16
4.1	Directory Structure	16
4.2	ROS Service and Action Definitions	17
4.2.1	Text-to-Speech Service	17
4.2.2	Text-to-Speech Action	18
4.3	Configuration Interface	19
4.4	External Script Interface	19
4.5	Error Handling	19
5	Module Design	21
5.1	Overall Architecture	21
5.2	Language Management	21
5.3	Configuration Management	22
5.4	Module Initialization	23
5.5	English TTS Implementation	24

5.6	Kinyarwanda TTS Implementation	25
5.7	Playback Implementation	25
5.8	Service Handler Implementation	25
5.9	Action Server Implementation	26
5.10	Output Suppression	27
5.11	Paths and Configuration	27
6	User Manual	28
6.1	Installing Dependencies	28
6.2	Building the Workspace	28
6.3	Configuring the Node	28
6.4	Launching the Node	29
6.5	Invoking the TTS Service	29
6.6	Invoking the TTS Action	29
6.7	Common Issues	30
6.7.1	NAOqi Driver Connection Failure	30
6.7.2	ModuleNotFoundError for torch or TTS	30
6.7.3	Local Playback Has No Sound	30
7	Unit Testing	31
7.1	Test Suite Organization	31
7.2	Service Interface Tests (Tests 1–6)	31
7.3	Action Server Interface Tests (Tests 7–11)	32
7.4	Test Results: Full Run with <code>interface = both</code>	32
7.5	Issues Encountered and Fixes Applied	33
7.6	Configuration Used for the Test Run	34
	References	35
	Principal Contributors	36
	Document History	37

1 Introduction

The integrated TTS module developed in this deliverable combines the capabilities of both English and Kinyarwanda text-to-speech conversion into a single unified system. This integration allows for seamless switching between languages, enabling the Pepper robot to communicate effectively in multilingual environments.

The basic workflow of the integrated TTS system follows this pattern: Text input is received either through a ROS service call or an action goal sent to `/textToSpeech/say_text`. Both interfaces share the same name and the choice between them is controlled by the `interface` parameter in the configuration file. The action interface additionally publishes incremental *feedback* (Synthesizing, Playing audio, Completed) so that callers can monitor progress and preempt the goal if needed.

For each request, the appropriate neural model synthesises the speech on the host machine: a fine-tuned YourTTS model for Kinyarwanda and an XTTS v2 model for English. The generated waveform is written to a temporary WAV file, which is then routed to one of two playback back-ends selected by the `playback_mode` parameter. In `naoqi` mode, the WAV file is copied to the robot via SSH/SCP and played through NAOqi's `ALAudioPlayer`; in `local` mode, it is played on the host machine via `paplay`, which is convenient for development and testing without a robot. The temporary WAV file is removed after playback.

A success response is returned to the service caller, or a result message (with `success`, an explanatory message, and the path to the generated WAV file) is returned to the action client.

This dual-path architecture leverages neural TTS for both languages while abstracting playback behind a configuration switch, so the same node can be exercised end-to-end on a developer laptop and then re-targeted at the robot without code changes. The implementation also bridges modern Python 3 TTS frameworks with the Python 2-only NAOqi SDK by delegating on-robot playback to a small Python 2 helper script.

This deliverable is structured as follows:

Section 2 outlines the system's functional and design considerations for the dual-language text-to-speech system. It details the service-based and action-based architectures and language-specific processing approaches. **Section 3** presents the module specification, detailing the text-to-speech conversion process for both English and Kinyarwanda languages. It explains how the system uses XTTS v2 for English and a fine-tuned YourTTS model for Kinyarwanda, and how playback is routed either to the robot via NAOqi or to the local host. **Section 4** describes the interface design, covering the ROS service and action definitions, the configuration file format, and the bridge mechanism between Python 3 and Python 2 environments. It details how the system handles text input through both the service and action interfaces and manages audio output through the NAOqi and local playback back-ends. **Section 5** details the module's architecture, including the split between the application entry-point (`text_to_speech_application.py`) and the core implementation (`text_to_speech_implementation.py`), the configuration loading and CUDA-fallback logic, model initialisation, request dispatch, preemption handling, output suppression, and the heartbeat timer. **Section 6** contains a user manual with step-by-step instructions for configuration and usage, including invoking the text-to-speech functionality through either the service or the action client. **Section 7** provides an overview of the testing procedures, including unit tests for each language path on both the service and action interfaces, and system validation under various network conditions and text input scenarios.

2 Requirements Definition

2.1 Overview

The objective of the integrated Text-to-Speech (TTS) system is to develop a software module that enables the robot to convert text input in either English or Kinyarwanda into natural-sounding speech. The system identifies the target language, processes the text accordingly, and generates appropriate audio output for playback on the robot's speakers. This TTS functionality serves as a crucial component for robot communication, enabling verbal interaction with users in multilingual contexts.

2.2 Functional Specification

The integrated TTS system operates as a ROS node called `textToSpeech` and provides the robot with the ability to vocalize text content in multiple languages. It supports two primary modes of operation, both implemented as neural-model synthesis on the host machine:

1. **English Mode:** Uses an XTTS v2 model (`TTS.tts.models.xtts.Xtts`) to synthesise speech from English text, conditioned on a short reference WAV for voice characteristics.
2. **Kinyarwanda Mode:** Uses a fine-tuned YourTTS model (loaded through `TTS.utils.synthesizer.Synthesizer`) to synthesise speech from Kinyarwanda text.

The system switches between these modes based on the `language` field carried by each request. Once synthesis is complete, the resulting waveform is written to a temporary WAV file and dispatched to one of two playback back-ends, controlled by the `playback_mode` configuration parameter: `naoqi` routes the audio to the robot through NAOqi's `ALAudioPlayer`, while `local` plays the audio on the host machine via `paplay` for testing without a robot. Independently of language and playback mode, the node can expose the text-to-speech functionality through either a ROS *service* interface, a ROS *action server* interface, or both simultaneously, depending on the value of the `interface` parameter. The action interface is preferred for long utterances or workflows where the caller needs progress feedback or the option to preempt playback; the service interface remains available for compact, blocking call sites.

2.3 Text Processing and Speech Synthesis

The system performs two key processes:

- **Text Parsing and Preprocessing:** The system receives text input through either a ROS service interface (`/textToSpeech/say_text`) or, when configured, a ROS action server advertised on the same name. Both accept a text string and a language identifier, and process the text according to the selected language mode.
- **Speech Synthesis:** For English, the system uses the XTTS v2 model directly, conditioned on a short reference WAV (configurable via `englishSpeakerWav`, with a fallback to `conditioning_audio.wav`). For Kinyarwanda, it uses the fine-tuned YourTTS model through the high-level Coqui Synthesizer wrapper, also conditioned on `conditioning_audio.wav`. In both cases the resulting waveform is written to a temporary WAV file. Playback is then handled by the back-end selected by `playback_mode`:

`naoqi` mode invokes a Python 2 helper script that copies the WAV to the robot via SSH/SCP and plays it through `ALAudioPlayer`, while `local` mode plays it directly on the host with `paplay`. The temporary WAV file is removed after playback. When the action interface is in use, the server publishes feedback messages (`Synthesizing`, `Playing audio`, `Completed`) at the corresponding stages of this pipeline so that the client can track progress.

The subsystem incorporates error handling mechanisms to provide meaningful feedback when errors occur, particularly when loading either of the neural TTS models.

2.4 Configuration Management

The robot's speech system is configured through a dedicated configuration file (`text_to_speech_configuration.ini`) that specifies:

1. Default language selection (English or Kinyarwanda) used when a request omits the language field
2. ROS interface to expose (`service`, `action`, or `both`)
3. Audio playback mode (`naoqi` for routing through the robot, or `local` for testing on the host machine)
4. Verbose mode enabling or disabling for debugging purposes
5. Robot IP address and NAOqi communication port (used in `naoqi` playback mode)
6. GPU acceleration flag (`useCuda`); when `True` but no CUDA device is available, the node automatically falls back to CPU inference
7. File-system paths to the English (XTTS v2) and Kinyarwanda (YourTTS) model directories, and an optional speaker reference WAV for the English model

The system falls back to default configuration values if the configuration file cannot be read or processed, ensuring continuous operation even in the event of configuration errors.

2.5 Inputs and Outputs

2.5.1 Service Provided

The `textToSpeech` node provides a service for text vocalization:

Service	Type	Description
<code>/textToSpeech/say_text</code>	<code>cssr_system/TTS</code>	Service that accepts a text message and a language identifier and returns a success status

Table 1: TTS Service Specification

2.5.2 Action Server Provided

When the `interface` configuration parameter is set to `action` or `both`, the `textToSpeech` node also advertises a ROS action server on the same name. The action interface accepts a text and a language identifier as the goal, and additionally publishes incremental feedback while the goal is being processed.

Action	Type	Description
/textToSpeech/say_text	cssr_system/TTSAction	Action server that accepts a goal containing a text field and a language field; publishes feedback (Synthesizing, Playing audio, Completed) with a numeric progress value, and returns a result containing a boolean success flag, an explanatory message string, and the path to the generated WAV file (audio_file_path)

Table 2: TTS Action Server Specification

2.6 External Dependencies and Integration

The TTS system relies on several external components for operation:

- **Coqui TTS library** (TTS package, version 0.22.0): provides both the high-level `Synthesizer` class used to load the Kinyarwanda YourTTS model and the lower-level `Xtts` class used for the English XTTS v2 model.
- **PyTorch** (`torch 2.5.1+cu121`, `torchaudio 2.5.1`): inference engine for both neural models, with optional CUDA acceleration when `useCuda = True` and a CUDA-capable GPU is available.
- **soundfile**: writes numpy waveform arrays to WAV files (used for the English path).
- **NAOqi Framework**: required only for `playback_mode = naoqi`, where audio is routed through the `ALAudioPlayer` proxy on the robot.
- **SSH/SCP via sshpass**: secure non-interactive file transfer to the robot for NAOqi-mode playback.
- **PulseAudio paplay**: required only for `playback_mode = local`, where audio is played on the host machine.

The system uses a Python 2 bridge script (`send_and_play_audio.py`) to interface with the NAOqi framework for audio playback, bridging the gap between the modern Python 3 ROS node and the Python 2 NAOqi SDK requirements. This script is only invoked in `naoqi` playback mode.

2.7 Error Handling and Logging

The system implements several mechanisms for error management:

- **Configuration errors**: Fallback to default values with appropriate warnings
- **Model loading errors**: Critical error logging and process termination if TTS models cannot be loaded
- **Operation feedback**: Logging of text being spoken and the selected language

- **Service response:** Boolean success indication in the service response

All errors and operational messages are logged through the ROS logging infrastructure to facilitate debugging and system monitoring.

3 Function specification

3.1 Functional Characteristics

3.1.1 Text-to-Speech Conversion

The integrated TTS module consists of a unified ROS node (`textToSpeech`) that handles text-to-speech conversion for both English and Kinyarwanda languages. The language is taken from each request's `language` field; when that field is empty, the default specified by the `language` key in `text_to_speech_configuration.ini` is used.

For English text, the system uses the XTTS v2 model (`TTS.tts.models.xtts.Xtts`) to synthesise the waveform on the host machine. For Kinyarwanda text, it uses a fine-tuned YourTTS model loaded through the higher-level `TTS.utils.synthesizer.Synthesizer` class. In both cases the resulting waveform is written to a temporary WAV file and dispatched to the playback back-end selected by `playback_mode` (NAOqi on the robot, or local on the host).

3.1.2 Operation Modes

The system supports two operation modes. In Normal Mode, standard operation occurs where text is converted to speech without additional output. In Verbose Mode, extended operation provides additional information logged for monitoring and debugging purposes; in this mode the verbose model output and NAOqi messages are no longer suppressed and become visible in the terminal.

3.1.3 Audio File Generation and Playback

For both English and Kinyarwanda TTS, the workflow is the same: the appropriate neural model synthesises the waveform on the host machine, the result is written to a temporary WAV file, and the file is then handed to the playback back-end. In `naoqi` mode, the WAV file is copied to the robot using SCP via a Python 2 helper script and played using the NAOqi `ALAudioPlayer` [1, 2]. In `local` mode, the WAV file is played directly on the host machine using `paplay`, which is convenient for development and testing without a robot. In either case, the temporary WAV file is removed after playback.

3.1.4 ROS Integration

The module operates as a ROS node named `textToSpeech`. Depending on the value of the `interface` configuration parameter, this node provides a service `/textToSpeech/say_text`, a ROS action server advertised on the same name, or both simultaneously. Both interfaces handle text-to-speech requests with explicit language specification and dispatch the request to the same underlying synthesis-and-playback pipeline.

3.2 Inputs and Outputs

The TTS module receives input text via a ROS service call, with the following specifications:

3.2.1 Inputs

The system receives input through either a ROS Service or a ROS Action Server, both advertised at `/textToSpeech/say_text`. The service uses the custom TTS service type, whose request carries

Parameter	Specification
Format	UTF-8 encoded text string
Maximum Length	1000 characters
Languages Supported	English, Kinyarwanda
Special Characters	Supported based on language requirements
Input Method	ROS service calls

Table 3: Input Text Specifications

a message field and a language field. The action interface uses the corresponding `TTSAction` action type, whose goal carries a `text` field (containing the string to synthesise) and a `language` field. Both interfaces accept text strings to be converted to speech in either English or Kinyarwanda. Additionally, the system reads from a configuration file `text_to_speech_configuration.ini`, which contains several parameters. These include the default language (english or kinyarwanda), the interface parameter (service, action, or both), the `playback_mode` parameter (naoqi or local), the `verboseMode` setting, the `useCuda` flag, the robot's IP address and port number, and file-system paths to the English and Kinyarwanda model directories.

3.2.2 Outputs

The system produces several types of output. The primary output is an audio waveform: regardless of language, the active neural model generates a temporary WAV file on the host machine, which is then either played locally on the host or transferred to the robot for playback through `ALAudioPlayer`. When the action interface is in use, the node also publishes `TTSAction` feedback messages (with status strings `Synthesizing`, `Playing audio`, and `Completed`, plus a numeric progress value in `[0.0, 1.0]`) and a final action result containing a boolean success flag, an explanatory message string, and the path to the generated WAV file in `audio_file_path` (the path is reported as a convenience and the file itself is deleted by the node before the result is returned). The system also provides logging through ROS logs, including a `textToSpeech: running heartbeat` published every 10 seconds for the lifetime of the node.

3.3 Dependencies

3.3.1 Common Dependencies

The system relies on several common dependencies. These include ROS Noetic (with `rospy` and `actionlib`), `rospkg` for ROS package path resolution, `pyyaml` required by ROS Python internals, the Coqui TTS library at version 0.22.0, PyTorch (`torch2.5.1+cu121` and `torchaudio2.5.1`) as the inference engine, `soundfile` for writing waveform arrays, and Python 3.9 (or compatible) on the host. `ConfigParser` from the Python standard library is used to load the INI configuration file. A Python 2 environment is additionally required only when `playback_mode = naoqi`, since the NAOqi SDK only exposes Python 2 bindings.

3.3.2 English TTS Dependencies

For English text-to-speech functionality, the system depends on the XTTS v2 model, loaded through `TTS.tts.models.xtts.Xtts`. The English model directory (`englishModelPath`, default `~/models/v2.0.2`) must contain four files: `config.json` (model configuration), `model.pth`

(model weights), `vocab.json` (tokeniser vocabulary), and `speakers_xtts.pth` (speaker embedding dictionary). A short reference WAV is used to condition the synthesised voice; the path is configurable via `englishSpeakerWav` and falls back to `conditioning_audio.wav` in the Kinyarwanda model directory when not set.

3.3.3 Kinyarwanda TTS Dependencies

The Kinyarwanda functionality uses the fine-tuned YourTTS model loaded through `TTS.utils.synthesizer.Synthesizer`. The Kinyarwanda model directory (`kinyarwandaModelPath`) must contain five files: `model.pth` (fine-tuned YourTTS weights), `config.json` (model architecture configuration), `speakers.pth` (speaker embedding dictionary), `SE_checkpoint.pth.tar` (speaker encoder checkpoint), and `config_se.json` (speaker encoder configuration) [2]. The synthesised speech is conditioned on `conditioning_audio.wav`, which acts as the reference speaker voice. When `playback_mode = naoqi`, on-robot playback additionally requires `sshpas` for non-interactive SSH/SCP and the NAOqi `ALAudioPlayer` on the robot.

3.4 Execution Workflow

The execution follows a defined workflow beginning with Node Initialization. During this phase, the application script (`text_to_speech_application.py`) sets file-system path class variables on `TTSImplementation` (the configuration file path, the model directory, and the path to the Python 2 audio helper), then registers a ROS node named `textToSpeech` with the master, instantiates `TTSImplementation`, and instantiates `TextToSpeechNode`. The implementation reads the INI configuration into `self.config`, performs an optional CUDA-availability check (falling back to CPU if a GPU was requested but is unavailable), loads the Kinyarwanda YourTTS model via `Synthesizer`, and loads the English XTTS v2 model via `Xtts.init_from_config` and `load_checkpoint`. `TextToSpeechNode` then advertises the ROS interface(s) selected by the interface configuration parameter: a ROS service, a ROS action server, or both. Once initialisation is complete, a ROS timer is registered that publishes `textToSpeech: running` every 10 seconds as a heartbeat, and `rospy.spin()` keeps the node alive.

Next in the workflow is Request Handling. When a service call is received, the handler reads the message and language fields, calls `say_text`, and returns a single success response after playback completes (the call is blocking). When an action goal is received, the handler reads the `text` and `language` fields and runs through four feedback stages: it publishes `Synthesizing` feedback, calls `synthesize` on the implementation, checks for client preemption, publishes `Playing` audio feedback, calls `play_audio`, publishes `Completed` feedback, and finally calls `set_succeeded` (or `set_aborted` on failure) with a `TTSResult` containing the success flag, an explanatory message, and the audio file path. In both cases the request information is logged.

The third phase involves Language-specific Synthesis. For English, the system invokes the XTTS v2 model with a speaker reference WAV (configurable, with a fallback to `conditioning_audio.wav`), `gpt_cond_len=3` for speaker conditioning, and `language="en"`; the resulting numpy waveform is written to a temporary WAV file via `soundfile.write`. For Kinyarwanda, the system calls the high-level `Synthesizer.tts` method with `conditioning_audio.wav` as the speaker reference, then writes the result via `Synthesizer.save_wav`. In verbose-disabled mode, the call is wrapped in an `_suppress_output` context manager that redirects `stdout` and `stderr` to `/dev/null` to silence verbose TTS-library logs.

The final phase is Audio Playback. In `naoqi` playback mode, the system invokes the Python 2 helper (`send_and_play_audio.py`) as a subprocess: the helper SCP-copies the WAV to `/home/nao/` on the robot using `sshpass`, calls `ALProxy("ALAudioPlayer", ip, port).playFile(...)`, and removes the file from the robot afterwards. In `local` playback mode, the system un-suspends the `PulseAudio` sink with `pactl` and then plays the WAV file directly using `paplay`. In both cases, the temporary host-side WAV file is deleted in a `finally` block regardless of success or failure.

3.5 System Requirements

The system has several requirements for proper operation. It requires a ROS Noetic workspace, a Python 3.9 (or compatible) environment for the main node with the Coqui TTS library and PyTorch installed, and pre-trained TTS models for both English (XTTS v2) and Kinyarwanda (fine-tuned YourTTS). For `naoqi` playback mode, an additional Python 2 environment with the NAOqi SDK, `sshpass` for non-interactive SSH/SCP, and network connectivity to the robot are required. For

`local` playback mode, a working PulseAudio installation with `paplay` is required on the host. GPU acceleration is optional and is enabled when both `useCuda = True` and a CUDA-capable GPU is detected by PyTorch.

3.6 Limitations and Assumptions

There are several limitations and assumptions in the current implementation. In `naoqi` playback mode, the system assumes the NAOqi driver is reachable and that SSH access to the robot is available with the password `nao`; the robot must also have sufficient disk space for temporary audio files. The action server can only be preempted before synthesis begins or between synthesis and playback — it cannot interrupt an in-progress `play_audio` call, which is a blocking subprocess. Audio quality may differ between the two languages due to different model architectures (XTTS v2 vs. YourTTS) and different speaker references. Currently, the system supports only two languages: English and Kinyarwanda.

4 Interface Design

4.1 Directory Structure

The integrated TTS module follows this directory structure:

```
cssr4africa/
├── cssr_system/
│   ├── text_to_speech/
│   │   ├── action/
│   │   │   └── TTS.action
│   │   ├── config/
│   │   │   └── text_to_speech_configuration.ini
│   │   ├── launch/
│   │   ├── model/
│   │   │   ├── model.pth
│   │   │   ├── config.json
│   │   │   ├── speakers.pth
│   │   │   ├── SE_checkpoint.pth.tar
│   │   │   ├── config_se.json
│   │   │   ├── conditioning_audio.wav
│   │   │   ├── conditioning_audio2.wav
│   │   │   └── README.md
│   │   ├── src/
│   │   │   ├── text_to_speech_application.py
│   │   │   ├── text_to_speech_implementation.py
│   │   │   └── send_and_play_audio.py
│   │   ├── srv/
│   │   │   └── TTS.srv
│   │   ├── action/
│   │   │   └── TTS.action
│   │   ├── text_to_speech_requirements.txt
│   │   └── README.md
│   ├── CMakeLists.txt
│   ├── package.xml
│   └── unit_tests/
│       ├── text_to_speech_test/
│       │   ├── src/
│       │   │   ├── text_to_speech_test_application.py
│       │   │   └── text_to_speech_test_implementation.py
│       │   └── README.md
```

The TTS module is split into an *application* script (`text_to_speech_application.py`), which contains the `_main_` entry-point and the `TextToSpeechNode` class wiring the ROS interfaces, and an *implementation* module (`text_to_speech_implementation.py`), which con-

tains `TTSImplementation` — the configuration loader, neural model initialisation, synthesis, playback, and cleanup logic. This separation keeps the ROS-facing surface and the TTS logic independently testable.

4.2 ROS Service and Action Definitions

The system exposes its text-to-speech functionality through two ROS interfaces sharing the same name (`/textToSpeech/say_text`). The `interface` configuration parameter selects which one(s) the node advertises at startup.

4.2.1 Text-to-Speech Service

The service interface allows for synchronous, blocking text-to-speech requests with explicit language specification:

```
# TTS.srv
string message # Input text to synthesize
string language # Language model to utilize
---
bool success # Operation status
```

This service is called as:

```
rosservice call /textToSpeech/say_text "message: 'Hello world' language:
'english'"
```

4.2.2 Text-to-Speech Action

The action interface accepts a goal containing a text string and a language identifier, and adds incremental feedback while the goal is being processed and a richer result on completion:

```
# TTS.action
# Goal
string text # Input text to synthesize
string language # Language model to utilize
---
# Result
bool success # Operation status
string message # Human-readable status (e.g. error description)
string audio_file_path # Path to the generated WAV file
---
# Feedback
string status # 'Synthesizing', 'Playing audio', 'Completed'
float32 progress # Progress in [0.0, 1.0]
```

A minimal Python action client looks like:

```
import rospy, actionlib
from cssr_system.msg import TTSAction, TTSGoal

rospy.init_node('tts_action_demo')
client = actionlib.SimpleActionClient('/textToSpeech/say_text', TTSAction)
client.wait_for_server()

goal = TTSGoal(text='Hello world', language='english')
client.send_goal(goal, feedback_cb=lambda fb: print(fb.status, fb.progress))
client.wait_for_result()
result = client.get_result()
print(result.success, result.message, result.audio_file_path)
```

Note that the service's input field is named `message` while the action goal's input field is named `text`; both carry the string to be spoken. Both interfaces share the same name, `/textToSpeech/say_text`, but are distinct ROS endpoints.

4.3 Configuration Interface

The system uses a simple INI-style configuration file with the following structure:

```
[DEFAULT]
language = english           # default language when request omits it
interface = both             # service, action, or both
playback_mode = naoqi       # naoqi (robot) or local (host)
verboseMode = False
ip = 172.29.111.240         # NAOqi robot IP (used in naoqi mode)
port = 9559                 # NAOqi robot port
useCuda = False             # use GPU if a CUDA device is available
englishModelPath = ~/models/v2.0.2
kinyarwandaModelPath =      # set at runtime if not specified
englishSpeakerWav =         # optional XTTS speaker reference WAV
```

The configuration is loaded at startup and affects the behavior of the system throughout its operation. The `language` key sets the default synthesis language used when a request does not specify one. The `interface` key controls which ROS interface(s) the node advertises (service, action, or both); set it to `both` to exercise the full service and action server test suite in a single run. The `playback_mode` key selects whether audio is routed through the NAOqi `ALAudioPlayer` on the robot (`naoqi`) or played locally on the host machine via `paplay` (`local`, useful for testing without a robot). The `useCuda` key requests GPU acceleration; if no CUDA device is detected at startup the node logs a warning and falls back to CPU. The `englishModelPath` and `kinyarwandaModelPath` keys point to the directories containing the XTTS v2 and YourTTS model files respectively, and `englishSpeakerWav` optionally overrides the speaker reference WAV used by XTTS v2 (the default is `conditioning_audio.wav` from the Kinyarwanda model directory).

4.4 External Script Interface

For `naoqi` playback mode, a Python 2 script is called with the following arguments:

```
python2 send_and_play_audio.py <audio_file_path> <robot_ip> <robot_port>
```

This script first suppresses NAOqi log output (unless `verboseMode = True`), then `scp`-copies the WAV file to `/home/nao/` on the robot using `sshpass`, calls `ALProxy("ALAudioPlayer", ip, port).playFile` to play it, and finally removes the file from the robot using `ssh rm`. Both the SCP and SSH calls use the hard-coded password `nao`. The script is invoked as a blocking subprocess by the implementation, and is independent of the synthesis language — it is used for any audio file generated by either model when `playback_mode = naoqi`.

4.5 Error Handling

The system provides error handling for the following scenarios:

1. **Configuration Errors:** If the INI file cannot be read or parsed, a warning is logged and the node continues with hard-coded default values.
2. **Model Loading Errors:** If either the YourTTS or the XTTS v2 model fails to load (missing files, corrupt checkpoints, etc.), an error is logged and the node exits with code 1.

3. **CUDA Fallback:** If `useCuda = True` but no CUDA-capable GPU is available, a warning is logged and the node falls back to CPU inference.
4. **Service Request Errors:** Any exception raised during synthesis or playback is caught by `say_text`, which returns `(False, error_string, "")` to the service handler; the handler then returns `TTSResponse(success=False)` to the caller.
5. **Action Goal Errors:** Any exception raised during goal execution is caught by `_execute_action`, which calls `set_aborted` with a `TTSResult` carrying `success=False` and the error string in message.
6. **Language Selection Errors:** If an unsupported language is requested, the request is rejected with `success=False` on the service path or `set_aborted` on the action path.
7. **Cleanup:** The temporary WAV file is always deleted in a `finally` block, regardless of success, failure, or preemption. The `_safe_remove` helper silently ignores missing files and permission errors.

5 Module Design

5.1 Overall Architecture

The integrated Text-to-Speech system provides a flexible architecture that supports both English and Kinyarwanda languages through a unified pair of ROS interfaces. The implementation is split into two Python modules to keep the ROS-facing surface and the synthesis logic independently testable:

1. **Application script** (`text_to_speech_application.py`): contains the `__main__` entry-point and the `TextToSpeechNode` class. The entry-point sets file-system paths on `TTSImplementation` (via `TTSImplementation.set_paths`), initialises the ROS node, instantiates `TTSImplementation` and `TextToSpeechNode`, and calls `rospy.spin()`. `TextToSpeechNode` reads `config['interface']` and conditionally advertises a ROS service (`_handle_service`) at `/textToSpeech/say_text`, an action server (`_execute_action`) on the same name, or both. If the value is unrecognised, a warning is logged and the node falls back to service.
2. **Implementation module** (`text_to_speech_implementation.py`): contains `TTSImplementation`, which owns the configuration, both neural models, and the synthesis-and-playback pipeline. Its principal methods are `read_config`, `initialize_components` (which calls `_load_kinyarwanda_model` and `_load_english_model`), `synthesize`, `play_audio`, `say_text` (the high-level synthesise-then-play helper used by the service path), and the `_suppress_output` context manager.
3. **Language Handlers** (inside `TTSImplementation.synthesize`):
 - **English Handler**: invokes the XTTS v2 model directly through `TTS.tts.models.xtts.Xtts`, conditioned on a speaker reference WAV; writes the resulting numpy array to a temporary WAV file using `soundfile.write`.
 - **Kinyarwanda Handler**: invokes the fine-tuned YourTTS model through the high-level `TTS.utils.synthesizer.Synthesizer` wrapper, conditioned on `conditioning_audio.wav`; writes the result via `Synthesizer.save_wav`.
4. **Playback Back-ends** (inside `TTSImplementation.play_audio`):
 - **NAOqi mode**: invokes the Python 2 helper `send_and_play_audio.py` as a subprocess; the helper SCP-copies the WAV to the robot via `sshpass`, plays it via `ALProxy` (`"ALAudioPlayer", ip, port`).`playFile`, and removes the file from the robot.
 - **Local mode**: un-suspends the PulseAudio sink with `pactl` and then plays the WAV using `paplay` on the host.

5.2 Language Management

During initialization, the TTS node loads both language models so that they are ready for use without further loading delays. The actual language used for a given request is taken directly from the language field of the service request or action goal; if that field is empty, the default specified by the `language` key in the configuration file is used. The system also interacts with the BehaviorController and the `culturalLanguageBase` node, which together provide cultural context information including the preferred language for communication; the BehaviorController forwards that preference

into each TTS request, allowing for dynamic language switching during operation without restarting the node.

5.3 Configuration Management

`TTSImplementation.__init__` populates `self.config` with hard-coded defaults and then calls `read_config` to override them from the INI file:

```
def read_config(self):
    parser = ConfigParser()
    parser.read(self.config_file_path)
    section = 'DEFAULT'

    # String values
    for key in ('language', 'interface', 'playback_mode', 'ip', 'port',
                'englishModelPath', 'kinyarwandaModelPath',
                'englishSpeakerWav'):
        if parser.has_option(section, key):
            self.config[key] = parser.get(section, key)

    # Booleans
    self.config['verboseMode'] = (
        parser.get(section, 'verboseMode', fallback='False') == 'True')
    self.config['useCuda'] = (
        parser.get(section, 'useCuda', fallback='False').lower()
        in ('true', '1', 'yes', 'on'))
```

The default configuration values are:

```
self.config = {
    'language':          'english',
    'interface':         'service',
    'playback_mode':     'naoqi',
    'verboseMode':       False,
    'ip':                '172.29.111.240',
    'port':              '9559',
    'useCuda':           False,
    'englishModelPath':  os.path.expanduser('~/.models/v2.0.2'),
    'kinyarwandaModelPath': '', # set at runtime if not specified
    'englishSpeakerWav': '',
}
```

If the configuration file cannot be read or parsed, a warning is logged and these defaults are kept.

5.4 Module Initialization

The application script's `__main__` block first injects the file-system paths onto `TTSImplementation` as class variables (allowing tests to override them) and then starts the node:

```
if __name__ == '__main__':
    current_file_dir = os.path.dirname(os.path.abspath(__file__))
    parent_dir       = os.path.dirname(current_file_dir)

    TTSImplementation.set_paths(
        config_file_path = os.path.join(parent_dir, "config",
                                         "text_to_speech_configuration.ini"),
        model_dir_path    = os.path.join(parent_dir, "model"),
        python2_script_path = os.path.join(current_file_dir,
                                             "send_and_play_audio.py"),
    )

    rospy.init_node('textToSpeech')
    tts_impl = TTSImplementation()
    node     = TextToSpeechNode(tts_impl)
    rospy.spin()
```

`TTSImplementation.__init__` populates the default config, calls `read_config`, and then runs `initialize_components`, which performs the CUDA-availability check and loads both neural models:

```
def initialize_components(self):
    if self.config['useCuda']:
        self.use_cuda = torch.cuda.is_available()
        if not self.use_cuda:
            rospy.logwarn("CUDA requested but not available. "
                          "Falling back to CPU.")
    else:
        self.use_cuda = False

    self._load_kinyarwanda_model()
    self._load_english_model()

    rospy.Timer(
```

```
rospy.Duration(HEARTBEAT_MSG_PERIOD),
lambda _: rospy.loginfo("textToSpeech: running"))
```

A 10-second heartbeat timer is registered after both models are loaded; it serves as a liveness indicator visible in the terminal and `rqt_console` for the lifetime of the node.

5.5 English TTS Implementation

The English model is XTTS v2, loaded directly via `TTS.tts.models.xtts.Xtts`:

```
def _load_english_model(self):
    model_dir = Path(self.config['englishModelPath']).expanduser()

    cfg = XttsConfig()
    cfg.load_json(str(model_dir / "config.json"))

    self.eng_model = Xtts.init_from_config(cfg)
    self.eng_model.load_checkpoint(cfg, checkpoint_dir=str(model_dir),
                                  eval=True)

    if self.use_cuda:
        self.eng_model.cuda()
```

The English model directory must contain four files: `config.json`, `model.pth`, `vocab.json`, and `speakers_xtts.pth`. Synthesis itself uses the lower-level `eng_model.synthesize` call:

```
outputs = self.eng_model.synthesize(
    text,
    cfg,
    speaker_wav=self._english_speaker_wav(),
    gpt_cond_len=3,
    language="en",
)
wav_array = outputs["wav"]

with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as fp:
    soundfile.write(fp.name, wav_array,
                    self.eng_model.config.audio.output_sample_rate)
    audio_file = fp.name
```

`_english_speaker_wav` returns `config['englishSpeakerWav']` when set, otherwise falls back to `conditioning_audio.wav` in the Kinyarwanda model directory. `gpt_cond_len=3` limits how many seconds of the reference audio are used for speaker conditioning.

5.6 Kinyarwanda TTS Implementation

The Kinyarwanda model is a fine-tuned YourTTS model loaded through the high-level Synthesizer wrapper:

```
def _load_kinyarwanda_model(self):
    model_dir = self._model_dir_path
    self.kin_synth = Synthesizer(
        os.path.join(model_dir, "model.pth"),
        os.path.join(model_dir, "config.json"),
        tts_speakers_file=os.path.join(model_dir, "speakers.pth"),
        encoder_checkpoint=os.path.join(model_dir, "SE_checkpoint.pth.tar"),
        encoder_config=os.path.join(model_dir, "config_se.json"),
        use_cuda=self.use_cuda,
    )
```

Synthesis is wrapped in `_suppress_output` so that verbose TTS-library logs are silenced unless `verboseMode = True`:

```
with self._suppress_output():
    wav = self.kin_synth.tts(
        text,
        speaker_wav=os.path.join(self._model_dir_path,
                                "conditioning_audio.wav"))

with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as fp:
    self.kin_synth.save_wav(wav, fp.name)
audio_file = fp.name
```

5.7 Playback Implementation

`play_audio` dispatches on the `playback_mode` configuration value:

```
def play_audio(self, audio_file):
    mode = self.config['playback_mode']
    if mode == 'naoqi':
        subprocess.run([
            '/usr/bin/python2',
            self._python2_script_path,
            audio_file,
            self.config['ip'],
            str(self.config['port']),
        ], check=True)
    elif mode == 'local':
        subprocess.run(['pactl', 'suspend-sink', 'RDPSink', '0'], check=False)
        subprocess.run(['paplay', '--volume=65536', audio_file], check=True)
    else:
        raise ValueError(f"Unknown playback_mode: {mode}")
```

Both branches block until playback has completed.

5.8 Service Handler Implementation

The service path is wired by `TextToSpeechNode._handle_service`, which delegates to `TTSImplementation.say_text` (a single high-level helper that synthesises, plays, and cleans

up):

```
def _handle_service(self, req):
    lang = req.language.lower() or self.tts_impl.config['language']
    success, message, _audio_path = self.tts_impl.say_text(req.message, lang)
    return TTSResponse(success=success)
```

`say_text` validates the language, calls `synthesize` to produce a temporary WAV file, calls `play_audio` to play it, and deletes the file in a `finally` block. The whole call is blocking — the service response is not sent until playback has finished.

5.9 Action Server Implementation

The action callback (`_execute_action`) runs through four feedback stages and additionally checks for client preemption between synthesis and playback. The status string transitions from Synthesizing (progress 0.2) to Playing audio (progress 0.7) to Completed (progress 1.0) immediately before the goal is set to succeeded:

```
def _execute_action(self, goal):
    feedback = TTSFeedback()
    result = TTSResult()
    audio_file = ""

    text = goal.text
    lang = goal.language.lower() or self.tts_impl.config['language']

    try:
        # Stage 1: Synthesizing
        feedback.status = "Synthesizing"
        feedback.progress = 0.2
        self._action_server.publish_feedback(feedback)

        if self._action_server.is_preempt_requested():
            self._action_server.set_preempted()
            return

        # Stage 2: synthesise (returns a temp WAV path)
        audio_file = self.tts_impl.synthesize(text, lang)

        if self._action_server.is_preempt_requested():
            self._safe_remove(audio_file)
            self._action_server.set_preempted()
            return

        # Stage 3: Playing audio
        feedback.status = "Playing audio"
        feedback.progress = 0.7
        self._action_server.publish_feedback(feedback)
        self.tts_impl.play_audio(audio_file)

        # Stage 4: Completed
        feedback.status = "Completed"
        feedback.progress = 1.0
        self._action_server.publish_feedback(feedback)

    result.success = True
```

```
result.message          = f"Spoken '{text}' in {lang}"
result.audio_file_path = audio_file
self._action_server.set_succeeded(result)

except Exception as exc:
    result.success        = False
    result.message        = str(exc)
    result.audio_file_path = ""
    self._action_server.set_aborted(result)

finally:
    self._safe_remove(getattr(result, 'audio_file_path', '') or audio_file)
```

The numeric progress field is non-decreasing within [0.0, 1.0] across the four stages. Action clients should treat goal completion (the `set_succeeded/set_aborted` transition signalled by the action result) as the definitive completion event; the Completed feedback message can occasionally be missed by a client because it is published immediately before the result and may race with it on the wire. Preemption is supported at two points: before synthesis begins (no work has been done yet) and between synthesis and playback (the temporary WAV file is deleted before the goal is cancelled). Preemption during playback itself is not supported, because `play_audio` is a blocking subprocess.

5.10 Output Suppression

`TTSImplementation._suppress_output` is a context manager that conditionally redirects `sys.stdout` and `sys.stderr` to `/dev/null` based on `verboseMode`:

```
@contextlib.contextmanager
def _suppress_output(self):
    if self.config.get('verboseMode', True):
        yield # pass-through
    else:
        old_stdout, old_stderr = sys.stdout, sys.stderr
        with open(os.devnull, 'w') as devnull:
            sys.stdout, sys.stderr = devnull, devnull
        try:
            yield
        finally:
            sys.stdout, sys.stderr = old_stdout, old_stderr
```

It wraps model loading and Kinyarwanda synthesis calls so that the verbose Coqui TTS library output is hidden from the user when `verboseMode = False`.

5.11 Paths and Configuration

`TTSImplementation` stores three file-system paths as class variables that are populated by the application's `_main__` block via `set_paths`:

```
class TTSImplementation:
    _config_file_path = "" # set by set_paths
    _model_dir_path   = "" # set by set_paths
    _python2_script_path = "" # set by set_paths

    @classmethod
```

```
def set_paths(cls, config_file_path, model_dir_path, python2_script_path):
    cls._config_file_path = config_file_path
    cls._model_dir_path = model_dir_path
    cls._python2_script_path = python2_script_path

supported_languages = ['english', 'kinyarwanda']
```

This indirection is what allows the test harness (`text_to_speech_test_implementation.py`) to instantiate `TTSImplementation` against test fixtures without modifying the production code.

6 User Manual

6.1 Installing Dependencies

The Python dependencies for the host-side TTS node are listed in `text_to_speech_requirements.txt` and can be installed with:

```
# Python 3.9 (recommended: dedicated virtual environment)
python3 -m venv venv
source venv/bin/activate
pip install -r src/cssr_system/text_to_speech/text_to_speech_requirements.txt

# Python 2 (only required for naoqi playback mode)
pip2 install pynaoqi

# System dependencies
sudo apt-get install sshpass pulseaudio-utils
```

6.2 Building the Workspace

```
cd ~/workspace/pepper_rob_ws
catkin_make
source devel/setup.bash
```

6.3 Configuring the Node

Edit the configuration file before launching the node:

```
nano src/cssr_system/text_to_speech/config/text_to_speech_configuration.ini
```

A typical configuration for testing on the host without a robot looks like:

```
[DEFAULT]
language = english
interface = both           # service, action, or both
playback_mode = local      # local for host, naoqi for robot
verboseMode = False
ip = 172.29.111.240
port = 9559
useCuda = False
```

```
englishModelPath = ~/models/v2.0.2
kinyarwandaModelPath =
englishSpeakerWav =
```

For deployment on the robot, set `playback_mode = naoqi` and ensure ip and port match the robot.

6.4 Launching the Node

If `playback_mode = naoqi`, first launch the NAOqi driver in a separate terminal:

```
roslaunch naoqi_driver naoqi_driver.launch nao_ip:=172.29.111.230 \
network_interface:=wlp0s20f3
```

Then start the TTS node itself:

```
source venv/bin/activate
source ~/workspace/pepper_rob_ws/devel/setup.bash
roslaunch cssr_system text_to_speech_application.py
```

The node will print a `textToSpeech`: running heartbeat every 10 seconds once both models are loaded.

6.5 Invoking the TTS Service

When interface is set to service or both, send a request via:

```
rosservice call /textToSpeech/say_text \
  "message: 'Hello, welcome to robotics lab.' language: 'english'"

rosservice call /textToSpeech/say_text \
  "message: 'Muraho, murakaza neza muri robotics lab.' language:
  'kinyarwanda'"
```

6.6 Invoking the TTS Action

When interface is set to action or both, a Python action client can be used:

```
import rospy, actionlib
from cssr_system.msg import TTSAction, TTSGoal

rospy.init_node('tts_action_demo')
client = actionlib.SimpleActionClient('/textToSpeech/say_text', TTSAction)
client.wait_for_server()

goal = TTSGoal(text='Hello, welcome to robotics lab.', language='english')
client.send_goal(goal,
  feedback_cb=lambda fb: rospy.loginfo(f"{fb.status} ({fb.progress:.2f})"))
client.wait_for_result()
print(client.get_result()) # success, message, audio_file_path
```

To restart the node after a configuration change:

```
# Press Ctrl+C in the node's terminal to stop, then re-run:  
roslaunch cssr_system text_to_speech_application.py
```

6.7 Common Issues

6.7.1 NAOqi Driver Connection Failure

Check that the robot IP is correct, ensure the network interface is correctly specified, and verify that the robot is powered on and connected to the network.

6.7.2 ModuleNotFoundError for torch or TTS

If the node fails to start with a `ModuleNotFoundError`, the wrong Python virtual environment is active. Activate the environment that has `torch 2.5.1+cu121` and `TTS 0.22.0` installed before sourcing the workspace.

6.7.3 Local Playback Has No Sound

In `local` playback mode, ensure that `PulseAudio` is running and that `paplay` is installed (`pulseaudio-utils` package). The node will still report `success=True` for goals when `paplay` returns successfully even if no audible output is produced; check that the host's audio output device is correctly configured.

7 Unit Testing

This report documents the testing of the integrated Text-to-Speech (TTS) system developed for the Pepper robot with support for both English and Kinyarwanda languages, exercised through both the ROS service and the ROS action server interfaces. The system was tested using a custom Python test harness running over the ROS testing framework, with all tests executed successfully.

The integrated TTS system provides language-specific speech synthesis capabilities for the Pepper robot. Key features include:

- English TTS using the XTTS v2 neural model with configurable speaker conditioning.
- Kinyarwanda TTS using the fine-tuned YourTTS model with conditioning audio.
- Two parallel ROS interfaces - service and action server - sharing the same name (/textToSpeech/say_text) and selectable through the interface configuration parameter.
- Two playback back-ends - NAOqi on the robot, and local on the host - selectable through the playback_mode configuration parameter.
- Optional CUDA acceleration with automatic CPU fallback when no GPU is available.

7.1 Test Suite Organization

The unit-test driver (`text_to_speech_test_application.py`) connects, at startup, independently to the ROS service and to the ROS action server. If either interface is unavailable (because the running node is configured for only one of them), the corresponding group of tests is skipped automatically rather than reported as a failure. The driver fails fast only when neither interface is reachable. Tests 1–6 exercise the service interface; Tests 7–11 exercise the action server interface and additionally verify the feedback sequence and the action result fields.

7.2 Service Interface Tests (Tests 1–6)

- **Test 1 – English TTS Integration (Service):** sends two English messages through the service and confirms via user audio confirmation.
- **Test 2 – Kinyarwanda TTS Integration (Service):** sends a Kinyarwanda message through the service and confirms via user audio confirmation.
- **Test 3 – Language Switching (Service):** alternates between English and Kinyarwanda within a single test session.
- **Test 4 – Empty Message (Service):** confirms that an empty input is handled gracefully.
- **Test 5 – Unsupported Language (Service):** confirms the service returns `success=False` for an unknown language identifier.
- **Test 6 – Long Message (Service):** confirms playback of a long utterance.

7.3 Action Server Interface Tests (Tests 7–11)

- **Test 7 – English TTS Integration (Action):** sends two English messages as action goals with user audio confirmation.
- **Test 8 – Kinyarwanda TTS Integration (Action):** sends a Kinyarwanda message as an action goal with user audio confirmation.
- **Test 9 – Action Feedback Sequence:** verifies that the server publishes the expected feedback statuses (`Synthesizing` → `Playing audio`) and that the progress values are non-decreasing in $[0.0, 1.0]$. The terminal `Completed` feedback is treated as optional because, in ROS actionlib, it is published immediately before `set_succeeded` and may race with the result message on the subscriber side; the action result itself is the definitive completion signal.
- **Test 10 – Action Result Fields:** verifies the action result contains a boolean `success` field and a non-empty message string.
- **Test 11 – Unsupported Language (Action):** verifies the action server rejects an unsupported language identifier with `success=False`.

7.4 Test Results: Full Run with `interface = both`

The test suite was run with the configuration parameters set to `interface = both`, `playback_mode = local`, and `verboseMode = True`, so that every interface and every test could be exercised in a single process. All 11 tests passed:

```
=====
TTS IMPLEMENTATION INTEGRATION TESTS
=====
textToSpeechTest: connecting to /textToSpeech/say_text (service and/or action).
This test will verify the TTS implementation via both the ROS service and
action server interfaces.
Tests for an interface are skipped automatically when that interface is not
advertised.
Make sure the TTS node is running before proceeding.

[TEST 1] English TTS Integration (Service) ..... PASSED
[TEST 2] Kinyarwanda TTS Integration (Service) ..... PASSED
[TEST 3] Language Switching (Service) ..... PASSED
[TEST 4] Empty Message (Service) ..... PASSED
[TEST 5] Unsupported Language (Service) ..... PASSED
[TEST 6] Long Message (Service) ..... PASSED
[TEST 7] English TTS Integration (Action) ..... PASSED
[feedback] Synthesizing (0.10)
[feedback] Playing audio (0.60)
[feedback] Completed (1.00)
[TEST 8] Kinyarwanda TTS Integration (Action) ..... PASSED
[feedback] Synthesizing (0.10)
[feedback] Playing audio (0.60)
[feedback] Completed (1.00)
[TEST 9] Action Feedback Sequence ..... PASSED
[TEST 10] Action Result Fields ..... PASSED
[TEST 11] Unsupported Language (Action) ..... PASSED
=====
```


TEST SUMMARY

```
=====
Total tests run (excluding skipped): 11
Passed: 11
Failed: 0
Skipped: 0
```

```
ALL IMPLEMENTATION TESTS PASSED! The TTS implementation is working correctly.
```

The summary table below maps each test to its interface and outcome:

#	Test	Interface	Result
1	English TTS Integration	Service	PASSED
2	Kinyarwanda TTS Integration	Service	PASSED
3	Language Switching	Service	PASSED
4	Empty Message	Service	PASSED
5	Unsupported Language	Service	PASSED
6	Long Message	Service	PASSED
7	English TTS Integration	Action	PASSED
8	Kinyarwanda TTS Integration	Action	PASSED
9	Action Feedback Sequence	Action	PASSED
10	Action Result Fields	Action	PASSED
11	Unsupported Language	Action	PASSED

Table 4: Per-test interface and outcome for the `interface = both` run

7.5 Issues Encountered and Fixes Applied

Three issues were encountered during the test development cycle and resolved before the final run:

- Wrong Python virtual environment.** The original `cssr4africa.text_to_speech.env` environment was missing `torch` and `TTS`, causing a `ModuleNotFoundError` when starting the TTS node. The launch instructions were corrected to source the `reatimetts_ws/venv` environment, which contains `torch 2.4.0`, `TTS 0.22.0`, `rospkg`, and `rospy`.
- XTTS synthesis failure on the Test 9 phrase.** The original Test 9 message ("Testing feedback") triggered an `index out of range in self` error inside the XTTS v2 model — a known PyTorch tokeniser edge case for certain short phrases — which caused the action goal to be aborted. The Test 9 message was changed to "Hello world", which the model handles reliably.
- Completed feedback race condition.** The action server publishes the Completed feedback immediately before calling `set_succeeded`. Because both are ROS topic publishes, the result message could occasionally arrive at the client before the Completed feedback was processed by the subscriber callback, causing Test 9 to fail intermittently with a missing Completed status. The fix has two parts: (a) the test helper `_send_action_goal_and_wait` now sleeps for 1.0 s after `wait_for_result` returns, allowing in-flight feedback callbacks to fire; (b) Test 9 treats `Synthesizing` and `Playing` audio as required (they are emitted long before `set_succeeded` and represent the meaningful state transitions) but treats

Completed as optional — when it is absent, an informational note is logged and the test still passes, since the action result (`success=True`) is itself the definitive completion signal.

7.6 Configuration Used for the Test Run

To exercise all 11 tests in a single process, the following configuration values were used during the test session. They should be restored to their deployment defaults afterwards.

Parameter	Test value	Deployment default
<code>interface</code>	<code>both</code>	<code>service</code>
<code>playback_mode</code>	<code>local</code>	<code>naoqi</code>
<code>verboseMode</code>	<code>True</code>	<code>False</code>

Table 5: Configuration values used for the `interface = both` test run

References

- [1] Edresson Casanova, Julian Weber, Christopher Shulby, Arnaldo Candido Junior, Eren Gölge, and Moacir Antonelli Ponti. Yourtts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone. *Proceedings of the 39th International Conference on Machine Learning*, pages 2709–2720, 2022.
- [2] Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. *Proceedings of the 38th International Conference on Machine Learning*, pages 5530–5542, 2021.

Principal Contributors

The main authors of this deliverable are as follows (in alphabetical order).

David Vernon, Carnegie Mellon University Africa.

Richard Muhirwa, Carnegie Mellon University Africa.

Tsegazeab Tefferi, Carnegie Mellon University Africa

Document History

Version 1.0

First draft.
Richard Muhirwa.
24 March 2025.

Version 1.1

Updated the section 4 Interface Design subsection Directory structure. Added parent folder `cssr4africa` and `cssr_system` and I include the internal `README.md` file.
Richard Muhirwa.
17 April 2025.

Version 1.2

I updated the unit testing section by adding more tests including test audio playback and test TTS service.
Richard Muhirwa.
25 April 2025.

Version 1.3

Added documentation of the new ROS *action server* interface for `textToSpeech`, alongside the existing service interface, and of the new `interface` and `playback_mode` configuration parameters. Updated the executive summary, introduction, functional specification, inputs/outputs (added `TTSAction` table), interface design (added `TTS.action` definition and an action client snippet), module design (updated `read_config`, module initialization, and added an Action Server Implementation subsection), the directory structure (added `action/TTS.action` and the detailed `unit_tests/text_to_speech_test/` layout), and the User Manual (action client invocation example, new INI keys). Replaced the unit-testing section with the new 11-test suite (Tests 1–6 service, Tests 7–11 action), the corresponding sample output for an `interface = both` run.
Richard Muhirwa.
8 May 2026.